

算法作业一——编程题文档

姓名：邹世豪 学号：2452865

第一题 四阶四段龙格-库塔法

1. (a)

设输入节点序列为 $\{x_k\}_{k=0}^N$ ，步长为 h 。利用标准四阶四段龙格-库塔公式时，从 x_0 到 x_N 需要进行 N 次迭代，每一步都要计算 4 个中间量 k_1 、 k_2 、 k_3 、 k_4 ，并用它们的线性组合得到下一步的数值解 y_{n+1} 。由于每一步中的函数求值次数和四则运算次数都是常数，因此单步计算代价为 $\Theta(1)$ ，总时间复杂度为 $\Theta(N)$ 。

若程序只保留当前步的 x_n 、 y_n 以及 k_1 、 k_2 、 k_3 、 k_4 等少量中间变量，则辅助空间复杂度为 $\Theta(1)$ ；若为了输出所有节点上的数值解和误差而保存全部结果，则需要额外存储 $N+1$ 个节点信息，此时总存储空间复杂度为 $\Theta(N)$ 。因此，这一问可以明确写成：时间复杂度为 $\Theta(N)$ ，辅助空间复杂度为 $\Theta(1)$ ；若保存全部节点结果，则空间复杂度为 $\Theta(N)$ 。

2. (b)

题目给定的初值问题为 $y' = \cos x$ ， $y(0) = 0$ ，其真解为 $y = \sin x$ 。程序采用四阶四段龙格-库塔法，在区间 $[0, 2\pi]$ 上取步长 $h = 0.1$ 进行数值求解。由于 2π 不是 0.1 的整数倍，所以计算时前面的步长取 0.1，最后一步取剩余长度，从而保证计算节点能够准确到达区间右端点 2π 。

在全部计算节点上，记数值解为 y_i ，真解为 $\sin(x_i)$ ，则均方误差定义为 $MSE = (1/n) \cdot \sum (y_i - \sin(x_i))^2$ ，其中 n 表示参与统计的节点总数。按照这一公式统计本题所有节点误差，得到的均方误差约为 5.9220×10^{-16} 。这一结果已经非常接近双精度浮点数的舍入误差范围，说明四阶四段龙格-库塔法在本题给定方程上的求解精度很高。

进一步分析可以看到，数值解曲线与真解 $y = \sin x$ 基本重合，区间内的最大绝对误差约为 3.4730×10^{-8} ，终点 $x = 2\pi$ 处的数值解约为 -1.5042×10^{-9} ，与真解 0 非常接近。因此，这一问可以写成：程序成功完成了数值求解，且在全部节点上的 MSE 很小，算法结果正确、稳定、精度高。

```
Microsoft Visual Studio 调试窗  ×  +  ∨

3.900000000000    -0.6877661831    -0.6877661592    -0.0000000239
4.000000000000    -0.7568025216    -0.7568024953    -0.0000000263
4.100000000000    -0.8182771395    -0.8182771111    -0.0000000284
4.200000000000    -0.8715758027    -0.8715757724    -0.0000000303
4.300000000000    -0.9161659686    -0.9161659367    -0.0000000318
4.400000000000    -0.9516021069    -0.9516020739    -0.0000000331
4.500000000000    -0.9775301516    -0.9775301177    -0.0000000340
4.600000000000    -0.9936910381    -0.9936910036    -0.0000000345
4.700000000000    -0.9999232923    -0.9999232576    -0.0000000347
4.800000000000    -0.9961646434    -0.9961646088    -0.0000000346
4.900000000000    -0.9824526467    -0.9824526126    -0.0000000341
5.000000000000    -0.9589243080    -0.9589242747    -0.0000000333
5.100000000000    -0.9258147145    -0.9258146823    -0.0000000322
5.200000000000    -0.8834546864    -0.8834546557    -0.0000000307
5.300000000000    -0.8322674711    -0.8322674422    -0.0000000289
5.400000000000    -0.7727645144    -0.7727644876    -0.0000000268
5.500000000000    -0.7055403501    -0.7055403256    -0.0000000245
5.600000000000    -0.6312666598    -0.6312666379    -0.0000000219
5.700000000000    -0.5506855617    -0.5506855426    -0.0000000191
5.800000000000    -0.4646021956    -0.4646021794    -0.0000000161
5.900000000000    -0.3738766778    -0.3738766648    -0.0000000130
6.000000000000    -0.2794155079    -0.2794154982    -0.0000000097
6.100000000000    -0.1821625106    -0.1821625043    -0.0000000063
6.200000000000    -0.0830894057    -0.0830894028    -0.0000000029
6.2831853072    -0.0000000015    -0.0000000000    -0.0000000015

MSE = 5.922042401466778e-16

C:\Users\TewRabbit\source\repos\算法\x64\Debug\hw1.exe (进程 31656)已退出, 代码为 0
```

第二题 求不平整水池的最终水面高度

1.

设第 i 个坑的底部高度为 α_i , 最终统一水面高度为 H , 则第 i 个坑中所容纳的水量应为 $\max(0, H - \alpha_i)$ 。因此总水量满足方程: $\sum \max(0, H - \alpha_i) = M$ 。问题的本质, 就是在给定所有 α_i 和总水量 M 的条件下, 求解最终水面高度 H 。

算法思想如下:

1. 在当前高度集合中, 使用线性时间选择算法求出一个枢轴值 p (例如中位数)。
2. 将所有高度划分为三部分:
 1. 小于 p 的元素集合 L
 2. 等于 p 的元素集合 E
 3. 大于 p 的元素集合 G

3. 设当前已经确定会被水淹没的区域个数为 c ，这些区域的高度和为 s 。
若将这些已确定区域以及集合 L 中的所有区域统一抬高到高度 p ，所需水量为：

$$W = (c + |L|) \cdot p - (s + \sum_{x \in L} x)$$

分情况讨论：

1. 若 $M < W$ ，说明水量不足以把水面抬到 p ，则最终水面高度一定小于 p ，因此递归到集合 L 中继续求解。
 2. 若 $M = W$ ，说明水量恰好把所有低于 p 的区域灌到 p ，因此最终水面高度就是 p 。
 3. 若 $M > W$ ，说明水面高度高于 p 。这时集合 L 和集合 E 中的区域都已经确定在水面以下或恰处于水面处，可以把它们合并到“已处理区域”中，并将剩余水量 $M - W$ 递归到集合 G 中继续求解。
4. 当递归到当前集合为空时，说明所有区域已经被水淹没，此时最终水面高度满足： $cH - s = M$ 因此可直接得到 c

复杂度应为 $O(n)$

2.

对于该问题，任何正确算法都必须读取全部 n 个区域的高度。原因如下：

假设存在一个算法在运行过程中没有读取某个区域的高度 α_j 。

那么构造两组输入数据：它们在除 α_j 以外的所有位置都完全相同，仅改变 α_j 的取值：在第一组中令 α_j 非常大；在第二组中令 α_j 非常小。

这样两组输入的最终水面高度将不同，但由于算法从未读取 α_j ，它对这两组输入的执行过程完全相同，只能输出同一个结果，因此不可能在两种情况下都正确。

所以，任何正确算法都必须检查所有 n 个输入元素，故该问题存在时间复杂度下界： $\Omega(n)$

而本算法的时间复杂度为 $O(n)$ ，因此该算法在渐进意义下是最优算法。

3.

题目给出的数据为 $\{\alpha_i\} = \{0.2, 0.5, 0.1, 0.8, 1.5, 0.4, 0.9\}$, $M = 2$ 。先排序得: 0.1, 0.2, 0.4, 0.5, 0.8, 0.9, 1.5。随后逐层计算所需水量: 将第 1 个坑从 0.1 提升到 0.2 需要 0.1 单位水; 将前 2 个坑从 0.2 提升到 0.4 需要 0.4 单位水; 将前 3 个坑从 0.4 提升到 0.5 需要 0.3 单位水; 将前 4 个坑从 0.5 提升到 0.8 需要 1.2 单位水。总水量恰好为 $0.1 + 0.4 + 0.3 + 1.2 = 2.0$ 。

因此, 题目给定数据对应的最终统一水面高度为 $H = 0.8$ 。

```
Microsoft Visual Studio 调试控 × + v
Please input n: 7
Please input 7 pit heights:
0.2 0.5 0.1 0.8 1.5 0.4 0.9
Please input total water amount M: 2
Final water level = 0.800000
```

4.

取 $\{\alpha_i\} = \{1, 3, 2\}$, $M = 2$ 。排序后得到 1, 2, 3。先将最低的一个坑从 1 提升到 2, 需要 1 单位水, 此时还剩 1 单位水; 接下来前两个坑的当前高度都为 2, 用剩余的 1 单位水继续抬高这两个坑, 则还能上升 $1/2$, 因此最终统一水面高度应为 $H = 2.5$ 。

验证如下: 当 $H = 2.5$ 时, 各坑中的水量分别为 $2.5 - 1 = 1.5$, $2.5 - 2 = 0.5$, $\max(0, 2.5 - 3) = 0$, 总水量为 $1.5 + 0.5 + 0 = 2$, 恰好等于给定的 M 。说明算法结论正确。

```
Please input n: 3
Please input 3 pit heights:
1 3 2
Please input total water amount M: 2
Final water level = 2.500000
```

结论

第一题中, 四阶四段龙格-库塔法求解给定初值问题的时间复杂度为 $\Theta(N)$, 辅助空间复杂度为 $\Theta(1)$, 在题目给定测试方程上取得了很高精度, 所有节点的 MSE 约为 $5.9220 \times 10^{(-16)}$ 。第二题

中，我给出的是排序加分层灌水的实现思路，能够正确求出最终水面高度；对于题目给定样例，结果为 $H = 0.8$ ；对于自拟测试样例 $\{1, 3, 2\}$ ， $M = 2$ ，结果为 $H = 2.5$ 。若进一步从理论上优化，则本题可达到 $O(n)$ 的最优时间复杂度。